

Pretentious Self-Parsing and Self-Evaluation of λ -Calculus

Heewon Lee (20210543)

School of Computing, KAIST

October 15, 2026

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computatable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computatable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

String “ $(\lambda. 0)$ ”

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

String “ $(\lambda. 0)$ ” $\xrightarrow[\text{(OCaml)}]{\text{Tokenize}}$ [LPAR; LAM; NAT(0); RPAR]

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

$$\begin{array}{ccc} \text{String } "(\lambda. 0)" & \xrightarrow[\text{(OCaml)}]{\text{Tokenize}} & [\text{LPAR}; \text{LAM}; \text{NAT}(0); \text{RPAR}] \\ & \xrightarrow[\text{(OCaml)}]{\text{Encode tokens}} & \sqcup(\lambda. 0)\sqcup \end{array}$$

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

$$\begin{array}{ccc} \text{String } "(\lambda. 0)" & \xrightarrow[\text{(OCaml)}]{\text{Tokenize}} & [\text{LPAR}; \text{LAM}; \text{NAT}(0); \text{RPAR}] \\ & \xrightarrow[\text{(OCaml)}]{\text{Encode tokens}} & \llbracket (\lambda. 0) \rrbracket \\ & \xrightarrow{\text{Parser}} & \text{App}(\text{e_parse}, \llbracket (\lambda. 0) \rrbracket) =_{\beta} \ulcorner (\lambda. 0) \urcorner \end{array}$$

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

$$\begin{array}{lcl} \text{String } "(\lambda. 0)" & \xrightarrow[\text{(OCaml)}]{\text{Tokenize}} & [\text{LPAR}; \text{LAM}; \text{NAT}(0); \text{RPAR}] \\ & \xrightarrow[\text{(OCaml)}]{\text{Encode tokens}} & \sqcup(\lambda. 0)\sqcup \\ & \xrightarrow{\text{Parser}} & \text{App}(\text{e_parse}, \sqcup(\lambda. 0)\sqcup) =_{\beta} \ulcorner(\lambda. 0)\urcorner \\ & \xrightarrow{\text{Evaluator}} & \text{App}(\text{e_eval}, \ulcorner(\lambda. 0)\urcorner) =_{\beta} (\lambda. 0) \end{array}$$

Concrete and Abstract Syntax

Concrete and Abstract Syntax

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\text{Lam}(t)$	(Function)
$\text{App}(t, t)$	(Application)

(De Bruijn index: $\text{Var}(0)$ = Closest variable, $\text{Var}(1)$ = Next closest, ...)
e.g.) $\lambda x. \lambda y. x y = \lambda. \lambda. 1\ 0$

Concrete and Abstract Syntax

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\text{Lam}(t)$	(Function)
$\text{App}(t, t)$	(Application)

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$(\lambda. \langle \text{term} \rangle)$	(Function)
$((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

(De Bruijn index: $\text{Var}(0)$ = Closest variable, $\text{Var}(1)$ = Next closest, ...)

e.g.) $\lambda x. \lambda y. x y = \lambda. \lambda. 1 0$

Concrete and Abstract Syntax

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\text{Lam}(t)$	(Function)
$\text{App}(t, t)$	(Application)

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$(\lambda. \langle \text{term} \rangle)$	(Function)
$((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

(De Bruijn index: $\text{Var}(0)$ = Closest variable, $\text{Var}(1)$ = Next closest, ...)

e.g.) $\lambda x. \lambda y. x y = \lambda. \lambda. 1\ 0$

- 1 Represent each token
- 2 Represent each AST constructor
- 3 Parser `e_parse` from token list to AST
- 4 Evaluator `e_eval` from AST to λ -expression

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda / r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda / r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda / r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda / r a v. v \ n$
---------------------------------------	----------------------------

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
$\lfloor \text{LPAR} \rfloor$	$= \lambda / r a v. l$
$\lfloor \text{RPAR} \rfloor$	$= \lambda / r a v. r$
$\lfloor \text{LAM} \rfloor$	$= \lambda / r a v. a$
$\lfloor \text{NAT}(n) \rfloor$	$= \lambda / r a v. v n$
$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v n$
$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \lceil t \rceil$
$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \lceil t_1 \rceil \lceil t_2 \rceil$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda l r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda l r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda l r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda l r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v l a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v l a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v l a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ } fv \text{ } fl \text{ } fa = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda / r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda / r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda / r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda / r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Example: $\lambda x. \lambda y. x = \lambda. \lambda. 1$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda / r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda / r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda / r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda / r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Example: $\lambda x. \lambda y. x = \lambda. \lambda. 1$

$\lambda v / a. l$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda / r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda / r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda / r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda / r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Example: $\lambda x. \lambda y. x = \lambda. \lambda. 1$

$$\lambda v / a. l \ (\lambda v / a. l \)$$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda / r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda / r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda / r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda / r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Example: $\lambda x. \lambda y. x = \lambda. \lambda. 1$

$$\lambda v / a. l \ (\lambda v / a. l \ (\lambda v / a. v \quad))$$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\llbracket \text{LPAR} \rrbracket$	$= \lambda l r a v. l$
-------------------------------------	------------------------

$\llbracket \text{RPAR} \rrbracket$	$= \lambda l r a v. r$
-------------------------------------	------------------------

$\llbracket \text{LAM} \rrbracket$	$= \lambda l r a v. a$
------------------------------------	------------------------

$\llbracket \text{NAT}(n) \rrbracket$	$= \lambda l r a v. v \ n$
---------------------------------------	----------------------------

$\lceil \text{Var}(n) \rceil$	$= \lambda v / a. v \ n$
-------------------------------	--------------------------

$\lceil \text{Lam}(t) \rceil$	$= \lambda v / a. l \ \lceil t \rceil$
-------------------------------	--

$\lceil \text{App}(t_1, t_2) \rceil$	$= \lambda v / a. a \ \lceil t_1 \rceil \ \lceil t_2 \rceil$
--------------------------------------	--

Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} fv \ n & (t = \lceil \text{Var}(n) \rceil) \\ fl \ \lceil t' \rceil & (t = \lceil \text{Lam}(t') \rceil) \\ fa \ \lceil t_1 \rceil \ \lceil t_2 \rceil & (t = \lceil \text{App}(t_1, t_2) \rceil) \end{cases}$$

Example: $\lambda x. \lambda y. x = \lambda. \lambda. 1$

$$\lambda v / a. l \ (\lambda v / a. l \ (\lambda v / a. v \ (\lambda s z. s \ z)))$$

Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$ Scott encoding: match expressions

Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$ Scott encoding: match expressions

Constructor	Church	Scott
<code>nil</code>	$\lambda n c. n$	$\lambda n c. n$
<code>cons <i>h t</i></code>	$\lambda n c. c\ h\ (t\ n\ c)$	$\lambda n c. c\ h\ t$

Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$ Scott encoding: match expressions

Constructor	Church	Scott
<code>nil</code>	$\lambda n c. n$	$\lambda n c. n$
<code>cons h t</code>	$\lambda n c. c\ h\ (t\ n\ c)$	$\lambda n c. c\ h\ t$
<code>Var(n)</code>	$\lambda v\ l\ a. v\ n$	$\lambda v\ l\ a. v\ n$
<code>Lam(t)</code>	$\lambda v\ l\ a. l\ (t\ v\ l\ a)$	$\lambda v\ l\ a. l\ t$
<code>App(t_1, t_2)</code>	$\lambda v\ l\ a. a\ (t_1\ v\ l\ a)\ (t_2\ v\ l\ a)$	$\lambda v\ l\ a. a\ t_1\ t_2$

Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$ Scott encoding: match expressions

Constructor	Church	Scott
<code>nil</code>	$\lambda n c. n$	$\lambda n c. n$
<code>cons h t</code>	$\lambda n c. c\ h\ (t\ n\ c)$	$\lambda n c. c\ h\ t$
<code>Var(n)</code>	$\lambda v\ l\ a. v\ n$	$\lambda v\ l\ a. v\ n$
<code>Lam(t)</code>	$\lambda v\ l\ a. l\ (t\ v\ l\ a)$	$\lambda v\ l\ a. l\ t$
<code>App(t_1, t_2)</code>	$\lambda v\ l\ a. a\ (t_1\ v\ l\ a)\ (t_2\ v\ l\ a)$	$\lambda v\ l\ a. a\ t_1\ t_2$

- Church: Top-level v, l, a are passed to every subexpression (Bottom-up)
- Scott: Top-level v, l, a operate on the immediate subexpressions directly (Top-down)

Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$ Scott encoding: match expressions

Constructor	Church	Scott
<code>nil</code>	$\lambda n c. n$	$\lambda n c. n$
<code>cons h t</code>	$\lambda n c. c\ h\ (t\ n\ c)$	$\lambda n c. c\ h\ t$
<code>Var(n)</code>	$\lambda v\ l\ a. v\ n$	$\lambda v\ l\ a. v\ n$
<code>Lam(t)</code>	$\lambda v\ l\ a. l\ (t\ v\ l\ a)$	$\lambda v\ l\ a. l\ t$
<code>App(t_1, t_2)</code>	$\lambda v\ l\ a. a\ (t_1\ v\ l\ a)\ (t_2\ v\ l\ a)$	$\lambda v\ l\ a. a\ t_1\ t_2$

- Church: Top-level v, l, a are passed to every subexpression (Bottom-up)
- Scott: Top-level v, l, a operate on the immediate subexpressions directly (Top-down)
- For parsing and evaluating, **Scott encoding** is more suitable!

Self-Interpreter e_eval

- Interpreter e_eval must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.

Self-Interpreter `e_eval`

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$
- $\text{eval } \ulcorner t_1 t_2 \urcorner \text{ env} = (\text{eval } t_1 \text{ env}) (\text{eval } t_2 \text{ env})$

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$
- $\text{eval } \ulcorner t_1 t_2 \urcorner \text{ env} = (\text{eval } t_1 \text{ env}) (\text{eval } t_2 \text{ env})$

`eval_comb =  $\lambda ast \ lst. ast$`

`( $\lambda n. \text{lookup } lst \ n$ )`

(Var: Lookup)

`( $\lambda t. \lambda. \text{eval\_comb } t \ (\text{cons } (\text{Var } 0) \ lst)$ )`

(Lam: Extend env, evaluate)

`( $\lambda t_1 \ t_2. (\text{eval\_comb } t_1 \ lst) (\text{eval\_comb } t_2 \ lst)$ )`

(App: Evaluate each, apply)

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$
- $\text{eval } \ulcorner t_1 t_2 \urcorner \text{ env} = (\text{eval } t_1 \text{ env}) (\text{eval } t_2 \text{ env})$

`eval_comb =  $\lambda ast \ lst. ast$`

`( $\lambda n. \text{lookup } lst \ n$ )`

(Var: Lookup)

`( $\lambda t. \lambda. \text{eval\_comb } t \ (\text{cons } (\text{Var } 0) \ lst)$ )`

(Lam: Extend env, evaluate)

`( $\lambda t_1 \ t_2. (\text{eval\_comb } t_1 \ lst) (\text{eval\_comb } t_2 \ lst)$ )`

(App: Evaluate each, apply)

- `cons`: Push every index up by one

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$
- $\text{eval } \ulcorner t_1 t_2 \urcorner \text{ env} = (\text{eval } t_1 \text{ env}) (\text{eval } t_2 \text{ env})$

`eval_comb =  $\lambda ast \ lst. ast$`

`( $\lambda n. \text{lookup } lst \ n$ )`

(Var: Lookup)

`( $\lambda t. \lambda. \text{eval\_comb } t \ (\text{cons } (\text{Var } 0) \ lst)$ )`

(Lam: Extend env, evaluate)

`( $\lambda t_1 \ t_2. (\text{eval\_comb } t_1 \ lst) (\text{eval\_comb } t_2 \ lst)$ )`

(App: Evaluate each, apply)

- `cons`: Push every index up by one
- Recursion via *Y*-combinator

Self-Interpreter e_eval

- Interpreter `e_eval` must satisfy $\text{App}(\text{e_eval}, \ulcorner M \urcorner) =_{\beta} M$.
- Conceptually, $\text{eval} \in \text{AST} \times \text{Env} \rightarrow \lambda\text{-expr}$.
- $\text{eval } \ulcorner n \urcorner \text{ env} = (n\text{-th variable in env})$
- $\text{eval } \ulcorner \lambda x. t \urcorner \text{ env} = \lambda x. (\text{eval } t (x :: \text{env}))$
- $\text{eval } \ulcorner t_1 t_2 \urcorner \text{ env} = (\text{eval } t_1 \text{ env}) (\text{eval } t_2 \text{ env})$

`eval_comb =  $\lambda ast \text{ lst}. ast$`

`( $\lambda n. \text{lookup } \text{lst } n$ )`

(Var: Lookup)

`( $\lambda t. \lambda. \text{eval\_comb } t (\text{cons } (\text{Var } 0) \text{ lst})$ )`

(Lam: Extend env, evaluate)

`( $\lambda t_1 t_2. (\text{eval\_comb } t_1 \text{ lst}) (\text{eval\_comb } t_2 \text{ lst})$ )`

(App: Evaluate each, apply)

- `cons`: Push every index up by one
- Recursion via *Y*-combinator
- $\Rightarrow \text{e_eval} = \lambda ast. \text{eval_comb } ast \text{ nil}$

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis
- $\text{parse} \in \text{tok list} \rightarrow \text{AST} \times \text{tok list}$

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis
- $\text{parse} \in \text{tok list} \rightarrow \text{AST} \times \text{tok list}$

$$\text{parse} (\ulcorner \text{NAT}(n) \urcorner :: \text{lst}) = (\ulcorner \text{Var}(n) \urcorner, \text{lst})$$

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis
- $\text{parse} \in \text{tok list} \rightarrow \text{AST} \times \text{tok list}$

$$\begin{aligned}\text{parse } (\text{⌊NAT}(n)\text{⌋} :: \text{lst}) &= (\ulcorner \text{Var}(n) \urcorner, \text{lst}) \\ \text{parse } (\text{⌊LPAR}\text{⌋} :: \text{⌊LAM}\text{⌋} :: \text{lst}) &= \text{let } (t, \text{⌊RPAR}\text{⌋} :: \text{lst}') = \text{parse } \text{lst} \text{ in} \\ &\quad (\ulcorner \text{Lam}(t) \urcorner, \text{lst}')\end{aligned}$$

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis
- $\text{parse} \in \text{tok list} \rightarrow \text{AST} \times \text{tok list}$

$$\begin{aligned}\text{parse } (\text{⌊NAT}(n)\text{⌋} :: \text{lst}) &= (\ulcorner \text{Var}(n) \urcorner, \text{lst}) \\ \text{parse } (\text{⌊LPAR}\text{⌋} :: \text{⌊LAM}\text{⌋} :: \text{lst}) &= \text{let } (t, \text{⌊RPAR}\text{⌋} :: \text{lst}') = \text{parse lst in} \\ &\quad (\ulcorner \text{Lam}(t) \urcorner, \text{lst}') \\ \text{parse } (\text{⌊LPAR}\text{⌋} :: \text{lst}) &= \text{let } (t_1, \text{⌊RPAR}\text{⌋} :: \text{lst}') = \text{parse lst in} \\ &\quad \text{let } (t_2, \text{⌊RPAR}\text{⌋} :: \text{lst}'') = \text{parse lst}' \text{ in} \\ &\quad (\ulcorner \text{App}(t_1, t_2) \urcorner, \text{lst}'')\end{aligned}$$

Self-Parser e_parse

- Similar for parsing, though more complicated case analysis
- $\text{parse} \in \text{tok list} \rightarrow \text{AST} \times \text{tok list}$

$$\begin{aligned}\text{parse } (\text{⌊NAT}(n)\text{⌋} :: \text{lst}) &= (\ulcorner \text{Var}(n) \urcorner, \text{lst}) \\ \text{parse } (\text{⌊LPAR}\text{⌋} :: \text{⌊LAM}\text{⌋} :: \text{lst}) &= \text{let } (t, \text{⌊RPAR}\text{⌋} :: \text{lst}') = \text{parse } \text{lst} \text{ in} \\ &\quad (\ulcorner \text{Lam}(t) \urcorner, \text{lst}') \\ \text{parse } (\text{⌊LPAR}\text{⌋} :: \text{lst}) &= \text{let } (t_1, \text{⌊RPAR}\text{⌋} :: \text{lst}') = \text{parse } \text{lst} \text{ in} \\ &\quad \text{let } (t_2, \text{⌊RPAR}\text{⌋} :: \text{lst}'') = \text{parse } \text{lst}' \text{ in} \\ &\quad (\ulcorner \text{App}(t_1, t_2) \urcorner, \text{lst}'')\end{aligned}$$

- Implementation in GitHub repository!

Summary

- **Pure λ -calculus is capable of reasoning about itself!**

Summary

- **Pure λ -calculus is capable of reasoning about itself!**
- Representing arbitrary constructors: Church vs. Scott encoding

Summary

- **Pure λ -calculus is capable of reasoning about itself!**
- Representing arbitrary constructors: Church vs. Scott encoding
- Parsing & Evaluation: Relatively straightforward match expressions